Assignment (Phase- 1)
Compiler Construction
(CS F363) Group 13

BITS PILANI

Under the supervision of

Prof. J Jabez Christopher

Marmik Sapovadia	2023A7PS0057H
Harshal Shah	2023A7PS0055H
Arav Thakkar	2023A7PS1091H
Mokshil Shah	2023A7PS0139H
Akshat Kumar	2023A7PS0117H

Table of Contents

Context-Free Grammar (CFG) Specification

1. Introduction

For this Assignment, a minimal subset of the Modula-2 language has been formally specified using a Context-Free Grammar (CFG). The designed grammar supports:

- Variable Declaration
- Assignment Statements
- Arithmetic Expressions
- Relational Expressions
- Logical Expressions
- Conditional Statement (IF-ELSE)
- Loop Statement (WHILE)

Operator precedence has been maintained across five levels:

1. Logical (OR, AND)
2. Relational
3. Addition/Subtraction
4. Multiplication/Division
5. Parentheses and Unary NOT

2. Complete CFG

```
Program -> 'MODULE' identifier ';' 'BEGIN' StatementSequence 'END' identifier '.'
StatementSequence -> Statement ';' StatementSequence  | Statement ';' | ε
Statement -> VarDecl  | Assignment  | Conditional  | Loop

// Variable Declaration
VarDecl  -> 'VAR' identifier ':' Type
Type -> 'INTEGER' | 'BOOLEAN'

// Assignment Statement
Assignment  -> identifier ':=' Expr

// Conditional Statement
Conditional -> 'IF' Expr 'THEN' StatementSequence 'ELSE' StatementSequence
'END'

// Loop Statement
Loop  -> 'WHILE' Expr 'DO' StatementSequence 'END'

// Expressions
// Level 1: Logical
Expr  -> Expr 'OR' RelExpr  | Expr 'AND' RelExpr | RelExpr

// Level 2: Relational
RelExpr  -> ArithExpr RelOp ArithExpr | ArithExpr
RelOp  -> '<' | '>' | '=' | '<=' | '>=' | '<>'

// Level 3: Arithmetic (Add/Sub)
ArithExpr -> ArithExpr AddOp Term  | Term
AddOp  -> '+' | '-'

// Level 4: Arithmetic (Mul/Div)
Term -> Term MulOp Factor | Factor
MulOp  -> '*' | '/'

// Level 5: Base Expressions
Factor -> identifier  | number  | 'TRUE'  | 'FALSE'  | 'NOT' Factor  | '(' Expr ‘)’
```

Lexical Specification

1. Introduction

Lexical analysis identifies tokens in the source program.
Each terminal symbol in the CFG is mapped to a corresponding regular expression.

2. Identifiers, Numbers and String

Identifier	**Regular Expression**
String	(\"[^\"\n]*\" | \'[^\'\n]*\')
Identifier	[a-zA-Z][a-zA-Z0-9]*
Number	[0-9]+

3. Keywords

Token	**Regular Expression**
MODULE	MODULE
BEGIN	BEGIN
END	END
VAR	VAR
INTEGER	INTEGER
BOOLEAN	BOOLEAN
IF	IF

THEN	THEN
ELSE	ELSE
WHILE	WHILE
DO	DO
AND	AND
OR	OR
NOT	NOT
TRUE	TRUE
FALSE	FALSE

4. Operators

Token	**Regular Expression**
Assignment	:=
Addition	\+
Subtraction	-
Multiplication	*
Division	/
Equal	=
Not Equal	<>
Less Than	<
Greater Than	>
Less or Equal	<=
Greater or Equal	>=

5. Punctuations

Token	Regular Expression
Semicolon	;
Colon	:
Comma	,
Left Parenthesis	\(
Right Parenthesis	\)
Dot	\.

6. Ignored Elements

Token	Regular Expression
Whitespace	[\t\n]+
Comments	"(*"([^*]|*+[^)*])**+")”

Conclusion

A minimal subset of the Modula-2 programming language has been formally defined using:

- A complete Context-Free Grammar (CFG).
- A full lexical specification using regular expressions.
- Multi-level operator precedence.
- Structured program blocks.
- Control statements and expressions.